Casbin: A Graph-Based Approach for Efficient and Verifiable Cloud Authorization

Yang Luo, *Member, IEEE*, Qingni Shen, *Senior Member, IEEE*, and Zhonghai Wu

***Abstract*—With the increasing adoption of hybrid cloud, existing authorization systems struggle to meet the increasingly complex authentication and authorization requirements in cloud environments. This paper proposes Casbin, a high-performance and user-friendly authorization framework. Casbin adopts a graph model-based policy language called PERM Modeling Language (PML), which supports various common authorization models such as RBAC and ABAC. Casbin introduces multiple graph computation optimization techniques, enabling millisecond-level authorization decisions under a policy scale of millions. Furthermore, Casbin leverages Satisfiability Modulo Theories (SMT) solvers for automated formal verification of policies, improving policy reliability. Casbin also innovatively utilizes Graph Retrieval-Augmented Generation (Graph RAG) technology to assist in the semi-automated generation of high-quality authorization policies. Experiments show that compared with methods like OpenFGA, Cedar, and SpiceDB, Casbin's average policy evaluation latency is reduced by 81.7%, and throughput is increased by 4.6 times. On large-scale complex policy sets, Casbin's SMT-based formal verification achieves a 92.1% recall rate and 94.5% precision rate for violation detection, with verification time controlled within 143.2 seconds. User studies indicate that compared to OpenFGA, Cedar, XACML, etc., PML's policy completion time is shortened by 57.3%, and accuracy is increased by 16.4%.**

***Index Terms*—Security policy language, access control, formal verification, large language model.**

I. Introduction

ACCESS control is one of the foundations of computer security. Traditional access control models such as RBAC [1] and ABAC [2] have laid the groundwork for authorization management in enterprise information systems. However, with the development of cloud computing, these models increasingly reveal deficiencies in performance and flexibility when facing the challenges of massive users, resources, and policies in hybrid cloud scenarios [3]. In hybrid cloud scenarios, a single business system often involves multiple private and public cloud services, and traditional authorization methods struggle to effectively manage the complex permission relationships and respond quickly to frequently changing resources and access policies; meanwhile, millions of API calls per second during business peaks impose extremely high real-time requirements on authorization systems, and existing solutions struggle to achieve stable millisecond-level authorization decisions under high concurrency. Moreover, policy authoring, as one of the most challenging tasks in information security management, can lead to serious consequences such as unauthorized access or data leakage when errors occur [4]. Due to the complexity of security policies, even experienced security administrators find it difficult to write completely error-free policies.

To address these challenges, the industry has successively introduced various new authorization solutions, including Google's Zanzibar [5], AWS's Cedar [6], OpenFGA [7] and SpiceDB [8] inspired by Zanzibar, as well as policy languages Rego [9] and XACML [10]. However, these solutions still have shortcomings in performance or usability: OpenFGA, Rego, and others exhibit significantly increasing matching times as policy scale grows, making it difficult to meet the real-time authorization needs of large-scale high-concurrency systems; XACML has cumbersome syntax, and Rego's declarative syntax and unique data model also introduce a high learning curve [11], where excessively high policy authoring barriers tend to introduce security vulnerabilities.

This paper proposes Casbin, a high-performance and easy-to-use authorization framework. The core of Casbin is the adoption of a policy language PML (PERM Modeling Language) [12] based on the PERM (Policy-Effect-Request-Matcher) graph model [13]. Unlike traditional list-based or tree-based policies, Casbin uses graphs to model authorization relationships among entities. This not only better aligns with actual business semantics but also enables the use of efficient graph algorithms for permission reasoning, thereby significantly improving policy matching performance. The framework's expressive power (RBAC, ABAC, multi-tenancy) primarily stems from PML's matcher, attributes, and stub functions; the role of the graph is to provide a unified structured representation for various policies, upon which optimizations such as graph partitioning, compression, and indexing are built to accelerate large-scale evaluation. Additionally, Casbin supports a complete formal verification method based on SMT constraints that can automatically detect errors in policies, such as redundancies and contradictions, greatly improving the reliability of policy configurations.

Received 3 February 2025; revised 28 June 2026; accepted 29 June 2026. Date of publication 2 July 2026; date of current version 1 September 2026. This work was supported in part by the Industry-University-Research Innovation Fund for Chinese Universities under Grant 2025SZ011 and in part by the National Key Research and Development Program of China under Grant 2022YFB2703301. *(Corresponding author: Zhonghai Wu.)*

The authors are with the School of Software and Microelectronics, Peking University, Beijing 102600, China, also with the National Engineering Research Center for Software Engineering, Peking University, Beijing 100871, China, also with the Beijing Key Laboratory of Data Intelligence and Security (Peking University), Beijing 100871, China, and also with the Key Laboratory of High Confidence Software Technologies of Ministry of Education (Peking University), Beijing 100871, China (e-mail: luoyang@pku.edu.cn; qingnishen@ss.pku.edu.cn; wuzh@pku.edu.cn).

Digital Object Identifier 10.1109/TDSC.2026.3709803

Casbin also innovatively utilizes Graph RAG (Graph-based Retrieval-Augmented Generation) [14] technology to achieve semi-automated policy generation. By combining the inherent graph-based characteristics of PML with Graph RAG, Casbin achieves better results than simply combining large language models (LLMs) with policy languages. Our experiments on real-world datasets show that compared with the latest methods such as SpiceDB, OpenFGA, and Cedar, Casbin reduces the average policy evaluation latency by 81.7% and improves throughput by 4.6 times. On large-scale complex policy sets, Casbin's SMT-based formal verification achieves a violation detection recall of 92.1% and a precision of 94.5%, with verification time kept within 143.2 seconds. Furthermore, user studies show that compared with OpenFGA, Cedar, XACML, and others, PML reduces policy completion time by 57.3% and improves correctness by 16.4%. The code of this paper has been released on GitHub.[1] After the necessary privacy review, we also plan to make the relevant datasets publicly available for researchers.

The main contributions of this paper are as follows:

1) Designed an easy-to-use, efficient authorization policy language PML, whose expressive power for models such as RBAC and ABAC stems from matchers, attributes, and stub functions, and which is uniformly modeled as an authorization relationship graph;
2) Introduced various graph computation optimization techniques, such as graph partitioning and graph compression, enabling Casbin to achieve millisecond-level authorization decisions at the scale of millions of policies;
3) Leveraged SMT solvers for automated formal verification of policies, greatly improving policy reliability;
4) Innovatively utilized Graph RAG technology to assist in the semi-automated generation of high-quality authorization policies.

The remainder of this paper is organized as follows. Section II introduces related work. Section III presents the details of our methodology. Section IV presents the experimental results. Section V discusses our approach. Section VI concludes the paper.

II. Related Work

Access control is a fundamental topic in the field of computer security. Sandhu et al. [1] proposed role-based access control: the RBAC96 family of models, introducing concepts such as role hierarchies and constraints, which laid the foundation for modern RBAC models. Ferraiolo et al. [2] further standardized RBAC by defining a reference model and functional specification for RBAC.

With the development of enterprise information systems, it was found that traditional RBAC has deficiencies in flexibility and expressiveness. Therefore, attribute-based access control (ABAC) emerged and has been widely adopted in military, financial, and other domains. Hu et al. [4] systematically summarized the development history of ABAC, proposed a general ABAC model, and discussed research challenges in authorization management. However, the ABAC model itself does not provide a standard policy language, resulting in significant differences in ABAC implementations across different systems and poor interoperability.

XACML [10] attempted to address this problem through a general-purpose XML-based policy language. It defines structural elements such as `rules`, `policies`, and `policy sets`, as well as policy combining algorithms such as `deny-overrides` and `permit-overrides`, offering strong expressive power. However, XACML's syntax is relatively complex, posing significant challenges for policy management. Ponder [15] is a policy definition language for distributed systems that supports access control by providing authorization, delegation, information filtering, and self-restraint policies. Ponder also supports obligation policies with event-triggered condition-action rules for policy-based management of networks and distributed systems. However, it lacks support for arithmetic logic, multi-tenancy, external functions, and other features, resulting in insufficient flexibility. SPL [16] is a security policy language that supports expressing entities, their relationships, and comparisons of attributes and quantifiers from different policies. However, SPL lacks external function support, which is a major limitation for more flexible policy evaluation. It also relies heavily on policy rules, which is unnecessary in certain scenarios.

In recent years, with the development of cloud computing, large-scale multi-tenant environments have imposed new requirements on access control. Traditional centralized authorization management approaches struggle to cope with the challenges of massive users, resources, and policies. Therefore, some new authorization frameworks and policy languages designed for cloud environments have begun to emerge.

OpenStack Access Control (OSAC), proposed by Tang et al. in [3], provides a formal description for concepts in Keystone such as domains, tenants, and roles. It further proposes a domain trust extension to the extended OSAC to facilitate secure cross-domain authorization. This work mainly focuses on enhancing Keystone, which differs from the goal of a general-purpose authorization framework. Google proposed the Zanzibar [5] distributed authorization system, which can support billions of users. It adopts a namespace and tuple design similar to relational databases, combined with caching optimization techniques, greatly improving policy evaluation performance. However, Zanzibar's policy language Rego [9] has a high learning curve, and there is room for improvement in syntax conciseness and debugging convenience. OpenFGA [7] is an open-source implementation of the Zanzibar philosophy, supporting fine-grained resource and relationship modeling. However, it still lacks support for advanced features such as policy inheritance and compilation optimization. AWS proposed the Cedar [6] policy language, which functionally supports basic RBAC and ABAC models and provides limited support for contextual policies and multi-tenant environments, but does not offer features such as policy compilation optimization. SpiceDB [8] is another open-source authorization database inspired by Zanzibar. SpiceDB uses a tuple-based relational model to store permission data and performs authorization decisions through graph

[1] https://github.com/casbin/casbin

traversal, making it conceptually similar to the graph model approach in this paper. SpiceDB supports fine-grained relationship modeling and provides a caveats mechanism for conditional permission checks (similar to conditional attributes in ABAC). However, SpiceDB's graph model is implicit—its permission relationships are expressed through tuple storage rather than explicit graph structures, making it difficult to directly apply graph computation optimization techniques such as graph partitioning and graph compression. Furthermore, SpiceDB does not provide built-in formal verification mechanisms to automatically detect redundancies and conflicts in policies, nor does it support semi-automated policy generation. In contrast, Casbin explicitly models authorization relationships as graph structures, enabling full utilization of optimization techniques from the graph computing domain, and provides more comprehensive policy management capabilities through SMT verification and Graph RAG.

OPA (Open Policy Agent) [9] provides a flexible policy language Rego that allows developers to define detailed policies to control microservices, CI/CD pipelines, and more. OPA supports dynamic contextual policies and time constraints but lacks policy indexing, and its performance is tens of times slower than native languages [11], making it difficult to handle large-scale authorization requests. Cerbos [17] is designed with the philosophy of simplifying access control for APIs and microservices for developers. While Cerbos performs well in RESTful model support and contextual policies, it lacks support for advanced features such as role inheritance and policy optimization. Wu et al. proposed Access Control as a Service (ACaaS) in [18], providing a new cloud service for comprehensive fine-grained access control. This approach claims to support multiple access control models; however, there is no evidence that this approach is applicable to models other than RBAC. Furthermore, this work is highly dependent on the IAM provided by AWS, making it difficult to apply to other cloud services. The work proposed by Jin et al. in [19] defines a proper formal ABAC specification for Infrastructure as a Service (IaaS) and implements it in OpenStack. It includes two models: the operational model $IaaS_{op}$ and the administrative model $IaaS_{ad}$, providing fine-grained access control for tenants. However, this work does not support external functions or decision composition, which limits its flexibility in expressing customized models.

In summary, although existing authorization frameworks and policy languages provide certain solutions under their respective design goals and application scenarios, they still have some limitations: either the functionality is not comprehensive enough, lacking support for advanced features such as role inheritance, time constraints, and policy indexing; or the performance is poor, unable to meet the high-concurrency authorization needs of large-scale systems; or the usability is lacking, with high usage complexity. These issues limit their effectiveness and universality in practical applications. The graph model-based policy language and authorization framework proposed in this paper adopts graph-based optimization techniques, significantly improving policy matching speed and meeting the high-performance requirements in cloud environments. It also supports features such as SMT formal verification and LLM-assisted policy generation, lowering the barrier to use and improving the efficiency and correctness of policy authoring.

III. Methodology

A. Preliminaries

This section reviews the basic definitions of the PML (PERM Modeling Language) policy language [12]. PML is based on the PERM (Policy-Effect-Request-Matcher) model [13] and is a flexible and expressive language for specifying graph-based access control policies. PML consists of six core elements: $\mathcal{R}$ (Request), $\mathcal{P}$ (Policy), $\mathcal{R}_{\mathcal{P}}$ (Policy Rule), $\mathcal{M}$ (Matcher), $\mathcal{E}$ (Effect), and $\mathcal{S}$ (Stub Function). These elements are organized and associated in the form of a graph structure. Vertices in the graph represent various types of entities (such as subjects, objects, actions, etc.), and edges represent the relationships between them (such as allow, deny, inherit, etc.). A PML policy is essentially a characterization of this authorization relationship graph. It follows the principle of separation of concerns, clearly distinguishing policy logic (i.e., access control rules) from policy data (i.e., specific roles, permissions, etc.). This allows policy logic to be reused while permitting policy data to change flexibly. The syntax of PML can be formally defined as:

$$\mathcal{R} ::= \langle r, \mathbb{A}\rangle \mid r \in \mathbb{E} \wedge \mathbb{A} \subseteq \mathbb{F}$$

$$\mathcal{P} ::= \langle p, \mathbb{A}\rangle \mid p \in \mathbb{E} \wedge \mathbb{A} \subseteq \mathbb{F}$$

$$\mathcal{R}_{\mathcal{P}} ::= \{value_1, value_2, value_3, \ldots\} \mid value_i \in \mathbb{V}$$

$$\mathcal{M} ::= \langle f, \langle \vec{v}, \vec{c}, \vec{s}\rangle\rangle \to \{0,1\} \mid f \in \mathbb{M}$$

$$\mathcal{E} ::= \langle g, T_1, T_2, \ldots\rangle \to \{0,1\} \mid g \in \mathbb{G}$$

$$\mathcal{S} ::= \langle name, \mathbb{P}\rangle \mid name \in \mathbb{N} \wedge \mathbb{P} \subseteq \mathbb{F}$$

where $\mathbb{E}$, $\mathbb{F}$, $\mathbb{V}$, $\mathbb{M}$, $\mathbb{G}$, and $\mathbb{N}$ denote the entity set, attribute name set, attribute value set, matcher function set, effect function set, and stub function name set, respectively. $\vec{v}$, $\vec{c}$, and $\vec{s}$ denote the attribute value vector, constant vector, and stub function vector, respectively. T denotes an effect term, composed of a quantifier $\mathfrak{Q}$ and a condition function h.

The matcher $\mathcal{M}$ is responsible for matching the attributes of the request $\mathcal{R}$ and the policy $\mathcal{P}$, returning 0 or 1 to indicate match failure or success. A simple equality matcher can be defined as:

$$\mathcal{M}_{eq} = \langle\, \lambda\langle\vec{v}\rangle.\, \bigwedge_{i=1}^{n} (r.attr_i = p.attr_i),$$
$$\langle\langle r.attr_1, \ldots, p.attr_1, \ldots\rangle\rangle\,\rangle$$

The effect $\mathcal{E}$ determines whether to allow or deny the request $\mathcal{R}$ based on the matching results. An allow-override effect can be defined as:

$$\mathcal{E}_{ao} = \langle\lambda\langle T\rangle.\exists(T.\mathfrak{Q} = \exists \wedge T.h(\vec{v}, \vec{c}, \vec{s}) = (p.eft \mapsto allow)),$$
$$\langle\langle\exists, \langle\lambda\langle\vec{v}\rangle.(v_{eft} \mapsto allow), \langle\langle v_{eft}\rangle\rangle\rangle\rangle\rangle\rangle$$

PML supports attribute-based access control (ABAC) by default. It can also be extended to support role-based access control (RBAC) and multi-tenancy through the use of stub functions

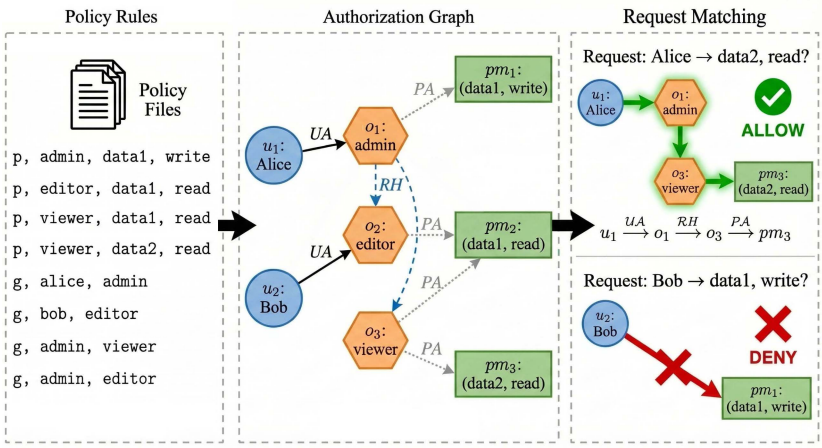


Fig. 1. Authorization graph request evaluation.

(such as $HasRole$ and $HasTenantRole$):

$$HasRole ::= \langle g, \{a_1, a_2\}\rangle \mid g \in \mathbb{N} \wedge a_1, a_2 \in \mathbb{F}$$

$$HasTenantRole ::= \langle g, \{a_1, a_2, t\}\rangle \mid g \in \mathbb{N} \wedge a_1, a_2, t \in \mathbb{F}$$

B. Graph-Based Policy Modeling

PML policies contain various types of entities (such as users, roles, tenants, permissions, etc.) and the associative relationships among them (such as user assignments, role inheritance, tenant roles, etc.). One of the core contributions of this paper is to explicitly organize these entities and relationships into an Authorization Graph. In this graph, vertices represent various types of entities and edges represent the relationships between them. Specifically, we define the authorization graph $G(V, E)$ as follows:

The authorization graph $G = (V, E, \lambda, \theta)$ is a labeled directed graph where the vertex set V consists of three types of entities: user set $\mathcal{U}$, role set $\mathcal{O}$, and permission set $\mathcal{PM}$, i.e., $V = \mathcal{U} \cup \mathcal{O} \cup \mathcal{PM}$. The edge set E consists of three types of relationships:

$$E = \{(u, o) \mid (u, o) \in \mathcal{UA}\} \cup \{(o_1, o_2) \mid (o_1, o_2) \in \mathcal{RH}\} \cup \{(o, pm) \mid (o, pm) \in \mathcal{PA}\}$$

where $\mathcal{UA} \subseteq \mathcal{U} \times \mathcal{O}$ is the user-role assignment relation, $\mathcal{RH} \subseteq \mathcal{O} \times \mathcal{O}$ is the role inheritance relation, and $\mathcal{PA} \subseteq \mathcal{O} \times \mathcal{PM}$ is the role-permission assignment relation. The edge label $\lambda : E \to \Pi$ carries ABAC attribute predicates (RBAC edges take $\top$, while edges carrying a `cond` take the predicate $\pi_{\texttt{cond}}(\vec{v}, \vec{c})$ evaluated by the matcher, and such an edge is activated in the reachability search only when it is true); the label $\theta : V \cup E \to \mathcal{T} \cup \{\bot\}$ marks tenants, and a request from tenant t only searches on the subgraph $G_t = (\{x \mid \theta(x) \in \{t, \bot\}\}, \{e \mid \theta(e) \in \{t, \bot\}\})$. The expressive power of predicates comes from PML's matcher and attribute mechanisms, while the responsibility of the graph is to provide unified carriage and traversal/partitioning acceleration. When no confusion arises, we abbreviate $G = (V, E, \lambda, \theta)$ as $G(V, E)$ in the following: edges without an explicitly annotated predicate take $\lambda(e) = \top$ by default, and vertices and edges without an explicitly annotated tenant take $\theta(\cdot) = \bot$ by default (i.e., not restricted by tenant).

It should be specifically noted that **policy rules themselves are not vertices in the graph**, but rather **induce edges and associated vertices in the graph**. Specifically, when loading a policy rule p_i, Casbin's policy parser extracts the relevant entities (users, roles, resources, etc.) from p_i as graph vertices and creates corresponding directed edges based on the authorization relationships expressed by p_i. For example, a policy rule `p, alice, data1, read` will induce the vertex `alice` (user) and the permission `(data1, read)`, as well as a directed edge from `alice` to that permission. Similarly, a role assignment rule `g, alice, admin` will induce a directed edge from user `alice` to role `admin`.

To more clearly illustrate how policy rules induce the construction of the authorization graph and how reachability analysis on the graph achieves authorization decisions, Fig. 1 provides a complete end-to-end example. This example demonstrates the entire process from raw policy rule text, through policy parsing and graph construction, to the final processing of a specific access request. The left side of the figure shows the input policy rule set, including permission assignment rules (`p`-type) and role assignment rules (`g`-type). The middle part of the figure shows the authorization graph $G(V, E)$ induced by these rules, which contains two user vertices (u_1: Alice, u_2: Bob), three role vertices (o_1: admin, o_2: editor, o_3: viewer), and three permission vertices (pm_1: (data1, write), pm_2: (data1, read), pm_3: (data2, read)). User-role assignment edges (solid lines), role inheritance edges (dashed lines), and role-permission assignment edges (dotted lines) are distinguished by different line styles. The right side of the figure shows the processing of the request "Can Alice read data2?": Casbin performs reachability analysis on the authorization graph, starting from vertex u_1 (Alice) and searching for a path along directed edges to reach the permission vertex pm_3 (data2, read). The existence of the highlighted path $u_1 \xrightarrow{\mathcal{UA}} o_1 \xrightarrow{\mathcal{RH}} o_3 \xrightarrow{\mathcal{PA}} pm_3$ indicates that the request is allowed. Meanwhile, the figure also shows that Bob's request "Can Bob write data1?" is denied because no directed path from u_2 (Bob) to pm_1 (data1, write) exists. This

contrast further demonstrates the intuitiveness and effectiveness of graph-based reachability judgment in authorization decisions.

This graph-based reachability judgment can be efficiently implemented through graph algorithms such as depth-first search (DFS) or breadth-first search (BFS). Based on this graph model, the policy matching process in PML can be uniformly understood as path searching and reachability analysis on the authorization graph. By analyzing the connectivity of the entities involved in a request within the graph, one can determine whether the request satisfies the defined access control policies. This graph-based approach enables PML to flexibly express complex authorization logic and support efficient policy evaluation.

It should be emphasized that the authorization graph can model not only RBAC-style roles and inheritance relationships but also naturally carry ABAC and multi-tenancy semantics under the same graph structure. For ABAC, attribute conditions are modeled as predicate labels attached to edges: a directed edge induced by a policy rule carrying a condition `cond`, in addition to recording the subject-object relationship, also carries an attribute constraint $\pi(\vec{v}, \vec{c})$, and only when the request context $\vec{v}$ satisfies π is the edge activated in the reachability search, thereby naturally integrating attribute matching into the graph traversal process. It should be noted that the flexibility of this attribute logic mainly derives from PML's matcher and attribute mechanisms, and the role of the graph abstraction is to provide it with efficient structured carriage and traversal acceleration, with the two complementing each other in responsibility. For multi-tenancy, each vertex and edge carries a tenant identifier t, and the authorization graph is accordingly partitioned into tenant-isolated subgraphs $\{G_{t_1}, \ldots, G_{t_k}\}$ (see graph partitioning in Section III-C); a request from tenant t_i only performs reachability search on the subgraph G_{t_i}, which both ensures permission isolation between tenants and reduces the search space.

Let $\mathcal{PM} = \{pm_1, \ldots, pm_m\}$ be the permission vertex set and $\mathcal{U} = \{u_1, \ldots, u_n\}$ be the user vertex set. It should be noted that the following reachability is defined relative to a given request context $\vec{v}$: an ABAC edge carrying a predicate label participates in the reachability computation only when $\lambda(e)$ evaluates to true under that context, while a pure RBAC edge ($\lambda(e) = \top$) is always activated. In this sense, the reachability between users and permissions can be represented as an $m \times n$ adjacency matrix A:

$$A_{ij} = \begin{cases} 1, & \text{if } u_j \text{ can reach } pm_i \text{ in } G \\ 0, & \text{otherwise} \end{cases}, 1 \le i \le m, 1 \le j \le n \tag{1}$$

The evaluation of request u_j against the permission set can be expressed as a matrix-vector product:

$$A\vec{e}_j = \vec{d} \tag{2}$$

where $\vec{e}_j$ is the j-th standard basis vector $(0, \ldots, 0, 1, 0, \ldots, 0)^\top$ with 1 at the j-th component; $\vec{d}$ is a binary decision vector $(d_1, \ldots, d_m)^\top$, where $d_i = 1$ indicates that u_j can reach pm_i and $d_i = 0$ indicates unreachable. The final decision $d^* \in \{0, 1\}$ for request u_j can be obtained by applying the effect expression $\mathcal{E}$ to the decision vector $\vec{d}$:

$$d^* = \mathcal{E}(\vec{d}) = \mathcal{E}(A\vec{e}_j) = \mathcal{E}\left(\sum_{i=1}^{m} A_{ij}\, \vec{e}_i\right) \tag{3}$$

Based on the graph model, permission checking in Casbin can be efficiently implemented using sparse matrix-vector multiplication and other linear algebra operations. Considering the sparsity of real-world policy sets, this approach can significantly reduce time and space overhead.

C. Enforcement Framework

Based on the PML policy language and the graph model described above, we designed the Casbin authorization framework, whose architecture is shown in Fig. 2. The core component `Enforcer` (identified in yellow) is responsible for enforcing access control decisions and consists of multiple submodules: `Matcher` handles matching between requests and policies, `Effector` determines the final access result, `Policy Rules` achieve persistent storage through the `Adapter`, and `Role Manager` manages role relationships and provides role information to the matcher.

The Casbin framework fully leverages the characteristics of the graph model, achieving efficient policy matching and permission reasoning through a series of optimization techniques. The following describes three core graph model optimization techniques in detail:

Graph Partitioning: In multi-tenant scenarios, Casbin partitions the authorization graph based on tenant IDs, ensuring strict isolation of policies between different tenants. Specifically, given an authorization graph $G(V, E)$ and a tenant set $\mathcal{T} = \{t_1, \ldots, t_k\}$, Casbin partitions G into k subgraphs $\{G_{t_1}, \ldots, G_{t_k}\}$, where $G_{t_i} = (V_{t_i}, E_{t_i})$ contains only the vertices and edges belonging to tenant t_i. This partitioning is performed statically at policy loading time: each policy rule is assigned to the corresponding subgraph based on its tenant identifier. When processing a request from tenant t_i, Casbin only performs reachability search on the subgraph G_{t_i}, thereby reducing the search space to $1/k$ of the original graph. When policies undergo dynamic updates (such as adding or deleting policy rules), only the subgraph of the corresponding tenant needs to be modified, without rebuilding the entire authorization graph.

Graph Compression: Casbin provides the `Adapter` module, which stores the adjacency matrix of the authorization graph using the compressed sparse row (CSR) format, significantly reducing memory usage. For an authorization graph with $|V|$ vertices and $|E|$ edges, traditional adjacency matrix storage requires $O(|V|^2)$ space, while the CSR format requires only $O(|V| + |E|)$. Since real-world authorization graphs are typically highly sparse (for example, in the ibm-loopback dataset, the non-zero elements of the adjacency matrix account for less than 0.1%), the CSR format yields significant memory savings.

Graph Indexing: To accelerate reachability queries, Casbin builds indexes for high-frequency query paths in the authorization graph. Specifically, Casbin maintains a hash table-based adjacency list to store the vertex and edge relationships in the

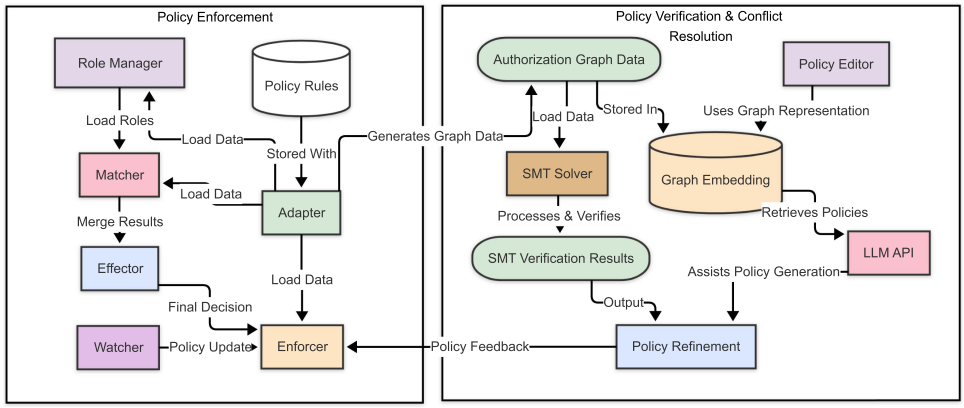


Fig. 2. Casbin authorization framework architecture.

graph. When new policies are loaded, Casbin parses them into edges in the graph and adds them to the adjacency list. For role inheritance relationships, Casbin precomputes and caches the transitive closure, enabling $O(1)$-time determination of whether an inheritance relationship exists between two roles during queries, avoiding redundant traversals for each query. When policies undergo incremental updates, Casbin does not rebuild the entire authorization graph or recompute the entire transitive closure but instead employs an incremental maintenance strategy: for a newly added edge (o_1, o_2), only o_2 and all its reachable successors are added to the reachable set of o_1; for edge deletion, a DFS-based reverse marking algorithm is used to backtrack only the affected reachability relationships. The average time complexity of this incremental maintenance is $O(d \cdot |V|)$ (where d is the average degree of affected vertices), which is far lower than the $O(|V|^3)$ required to rebuild the entire transitive closure, enabling Casbin to efficiently support scenarios with frequently changing policies.

Additionally, Casbin provides the `Watcher` module for real-time synchronization of policy updates in distributed environments. The `Watcher` uses technologies such as the Paxos consensus algorithm [20] to notify all registered `Enforcer` modules of policy changes, minimizing latency and network load to improve data synchronization efficiency.

The policy evaluation process in PML can be formalized as a function $\mathcal{S}$ that maps a request $\nabla \in \mathcal{R}$, a policy set $\mathcal{P}$, and a matcher $\mathcal{M}$ to a decision $d \in \{0, 1\}$:

$$\mathcal{S} : \mathcal{R} \times \mathcal{P} \times \mathcal{M} \to \{0, 1\} \tag{4}$$

where $\mathcal{S}(r, \mathcal{P}, \mathcal{M}) = 1$ indicates that the request $r \in \mathcal{R}$ is allowed by $\mathcal{P}$ under the constraints of $\mathcal{M}$, and $\mathcal{S}(r, \mathcal{P}, \mathcal{M}) = 0$ indicates that r is denied.

The execution of PML policies involves two core components: the matcher $\mathcal{M}$ and the effector $\mathcal{E}$. The matcher is responsible for evaluating the attribute matching between requests and policies, and the effector is responsible for combining the decisions of multiple matched policies. Role inheritance in RBAC does not require a separate role manager for formal characterization, as it is implicitly expressed by the transitive closure of the role inheritance subgraph $\mathcal{RH}$ in the authorization graph G—during evaluation, the reachability search starting from a user vertex automatically covers the permission vertices corresponding to all inheritable roles.

Algorithm 1 presents the pseudocode for PML policy evaluation. It first extracts attributes $\vec{v}$ from the request ∇ and the policy $\mathcal{P}$, and extracts constants $\vec{c}$, stub functions $\vec{s}$, and the matching expression $expr$ from the matcher $\mathcal{M}$ (lines 2-7). Here, $expr$ is the Boolean expression defined in the matcher $\mathcal{M}$, used to determine whether the specified matching conditions (such as equality matching, regular expression matching, or path matching) are satisfied between request attributes and policy attributes. Then, $expr$ is evaluated under the context $ctx = \langle \vec{v}, \vec{c}, \vec{s} \rangle$ to obtain the matching result (lines 8-9). If the match is successful, the algorithm traverses all policy rules and collects all policies $\vec{p}$ that match the request (lines 10-16). Subsequently, the RESOLVE_CONFLICTS function is called to perform conflict resolution on the multiple matched policies (line 17). This function determines the final authorization result from multiple potentially contradictory policy decisions based on the policy combination semantics defined in the effect expression $\mathcal{E}$ (such as "allow-override", "deny-override", etc.). The specific design of conflict resolution can be found in Section III-E. Finally, the algorithm extracts the effect attributes $\vec{v}_e$ from the matched policies $\vec{p}$, and evaluates the effect expression $\mathcal{E}^*$ under the effect context ctx_e to obtain the final decision (lines 18-20).

D. Formal Verification

To support more advanced policy analysis and ensure the correctness and security of access control systems, PML adopts a formal verification mechanism based on Satisfiability Modulo Theories (SMT). By encoding PML policies as SMT formulas, off-the-shelf SMT solvers (such as Z3 [21]) can be used to automatically check various properties, including consistency, redundancy, completeness, and security.

Formally, let Γ denote a set of PML policy rules, Ψ denote a single PML policy rule, Φ denote a constraint, and Ξ denote a domain knowledge axiom. Domain knowledge axioms refer to semantic constraints inherent to specific application scenarios, such as "a user cannot simultaneously hold the auditor and

Algorithm 1: PML Policy Evaluation.

Require: Request ∇, policy set $\mathcal{P}$, matcher $\mathcal{M}$
Ensure: Decision $d \in \{0, 1\}$
$\vec{v} \leftarrow \emptyset$, $\vec{c} \leftarrow \emptyset$, $\vec{s} \leftarrow \emptyset$
for $attr \in \nabla.\mathbb{A} \cup \mathcal{P}.\mathbb{A}$ **do** ▷ Extract attributes
  $\vec{v} \leftarrow \vec{v} \cup \{attr\}$
end for
for $f \in \mathcal{M}$ **do** ▷ Extract matcher components
  $\vec{c} \leftarrow \vec{c} \cup f.\vec{c}$, $\vec{s} \leftarrow \vec{s} \cup f.\vec{s}$
end for
$expr \leftarrow \mathcal{M}.expr$ ▷ Get matcher expression
$ctx \leftarrow \langle \vec{v}, \vec{c}, \vec{s} \rangle$ ▷ Build evaluation context
$match \leftarrow \texttt{EVAL}(expr, ctx)$ ▷ Evaluate matcher
if $match$ **then**
  $\vec{p} \leftarrow \emptyset$ ▷ Collect matched policies
  for $p \in \mathcal{P}$ **do**
    if EVAL_RULE$(\mathcal{M}, \nabla, p)$ **then**
      $\vec{p} \leftarrow \vec{p} \cup \{p\}$
    end if
  end for
  $\mathcal{E}^* \leftarrow$ RESOLVE_CONFLICTS $(\vec{p})$ ▷ Resolve policy conflicts
  $\vec{v}_e \leftarrow \bigcup_{p \in \vec{p}} p.\mathbb{A}$ ▷ Extract effect attributes
  $ctx_e \leftarrow \langle \vec{v}_e, \vec{c}, \vec{s} \rangle$ ▷ Build effect context
  $d \leftarrow \texttt{EVAL}(\mathcal{E}^*, ctx_{\mathbf{e}})$ ▷ Evaluate effect
else
  $d \leftarrow 0$ ▷ No match, return Deny
end if
return d

administrator roles" (mutual exclusion constraint) or "role inheritance relationships cannot form cycles" (acyclicity constraint). These axioms are independent of specific policy rules and reflect invariant properties of the business domain, used to provide additional semantic constraints in SMT verification. Thus, the SMT encoding of Γ can be defined as:

$$SMT(\Gamma) = \bigwedge_{\Psi \in \Gamma} SMT(\Psi) \wedge \bigwedge_{\Phi} SMT(\Phi) \wedge \bigwedge_{\Xi} SMT(\Xi) \quad (5)$$

where $SMT(\cdot)$ is a recursive function that converts PML expressions into equivalent SMT formulas according to the syntax and semantics of the policy language.

The SMT encoding of a policy rule Ψ can be further defined as:

$$SMT(\Psi) = \begin{cases} \top, & \text{if } \Psi = \langle r, \mathbb{A} \rangle \\ \bigwedge_{p=1}^{q} (r.a_p \bowtie v_p), & \text{if } \Psi = \{(a_1, v_1, \bowtie_1), \\ & \quad \ldots, (a_q, v_q, \bowtie_q)\} \\ SMT(\Psi_l) \oplus SMT(\Psi_r), & \text{if } \Psi = \langle \Psi_l, \Psi_r, \oplus \rangle \end{cases} \quad (6)$$

where $r.a_p \bowtie v_p$ denotes that the attribute a_p satisfies the relation $\bowtie$ (such as $=, \leq, \geq$, etc.) with value v_p, and $\oplus$ denotes a logical connective (such as $\wedge$, $\vee$, etc.). The SMT encoding of constraints Φ and domain axioms Ξ can be similarly defined.

Based on the above SMT encoding, we can leverage SMT solvers to automatically check various properties of PML policies, including the following four categories as well as extended properties common in cloud environments:

- *Consistency:* There are no contradictory rules in the policy set Γ, i.e., $SMT(\Gamma) \not\models \bot$.
- *Redundancy:* There are no superfluous rules in the policy set Γ, i.e., for any $\Psi_i \in \Gamma$, $SMT(\Gamma \setminus \{\Psi_i\}) \not\equiv SMT(\Gamma)$.
- *Completeness:* The policy set Γ covers all possible requests, i.e., for any $\nabla \in \mathcal{R}$, $SMT(\Gamma) \wedge SMT(\nabla) \models d$, where $d \in \{0, 1\}$.
- *Security:* The policy set Γ satisfies a given security property φ, i.e., $SMT(\Gamma) \models SMT(\varphi)$.
- *Mutual Exclusion Conflict:* Two roles are defined as mutually exclusive (e.g., specified through domain axiom Ξ), but the policy assigns the same user to both roles simultaneously. Formally, if Ξ contains $\forall u \in \mathcal{U} : \neg(HasRole(u, o_1) \wedge HasRole(u, o_2))$, then verifying the satisfiability of $SMT(\Gamma) \wedge SMT(\Xi)$ can detect such conflicts.
- *Cyclic Inheritance:* There exists a cycle in the role inheritance relationships, i.e., $\exists o_1, o_2, \ldots, o_k \in \mathcal{O} : (o_1, o_2) \in \mathcal{RH} \wedge \cdots \wedge (o_k, o_1) \in \mathcal{RH}$. This property is verified by encoding the role inheritance relationships as partial order constraints and checking their antisymmetry and transitivity.

For consistency checking, the Z3 solver can be used to determine whether $SMT(\Gamma)$ is satisfiable. If it is unsatisfiable, then contradictory rules exist. We can also consider using other solvers or engines, such as SAT solvers and theorem provers, which may have better performance in certain scenarios.

For security checking, it is necessary to first define the SMT constraint $SMT(\varphi)$ corresponding to the security property φ, and then determine whether $SMT(\Gamma) \models SMT(\varphi)$ holds. Taking the principle of least privilege as an example, one can define the property "for any employee role, write operations are not allowed":

$$\varphi = \forall r \in \mathcal{R} : (role(r) = employee \rightarrow action(r) \neq write)$$

Based on this, the SMT constraint can be constructed:

```
from z3 import *
def check_privilege(policy_set, user_role):
    s = Solver()
    for policy in policy_set:
        s.add(policy)
    s.add(user_role == 'employee')
    s.add(action == 'write')
    return s.check() == unsat
```

Regarding the SMT encoding of stub functions, a special note is needed. Stub functions (such as $HasRole$ and $HasTenantRole$) are essentially queries on role inheritance relationships in the authorization graph. In SMT encoding, we expand the semantics of stub functions into their equivalent logical constraints. Specifically, $HasRole(u, o)$ is encoded as a reachability constraint from user u to role o in the authorization

TABLE I
SMT ENCODING RULES FOR PML SYNTAX ELEMENTS

PML syntax element	Formalization	SMT encoding rule
Request	$\mathcal{R} ::= \langle r, \mathbb{A}\rangle \mid r \in \mathbb{E} \wedge \mathbb{A} \subseteq \mathbb{F}$	$SMT(\mathcal{R}) = \bigwedge_{a\in\mathbb{A}} (r.a = v_a)$
Policy	$\mathcal{P} ::= \langle p, \mathbb{A}\rangle \mid p \in \mathbb{E} \wedge \mathbb{A} \subseteq \mathbb{F}$	$SMT(\mathcal{P}) = \top$
Policy Rule	$\mathcal{R}_\mathcal{P} ::= \{v_1, v_2, v_3, \ldots\} \mid v_i \in \mathbb{V}$	$SMT(\mathcal{R}_\mathcal{P}) = \bigwedge_{i=1}^{n} (a_i = v_i)$
Matcher	$\mathcal{M} ::= \langle f, \langle \vec{v}, \vec{c}, \vec{s}\rangle\rangle \to \{0,1\} \mid f \in \mathbb{M}$	$SMT(\mathcal{M}) = f(\vec{v}, \vec{c}, \vec{s})$
Effect	$\mathcal{E} ::= \langle g, T_1, T_2, \ldots\rangle \to \{0,1\} \mid g \in \mathbb{G}$	$SMT(\mathcal{E}) = g(SMT(T_1), SMT(T_2), \ldots)$
Stub Function	$\mathcal{S} ::= \langle name, \mathbb{P}\rangle \mid name \in \mathbb{N} \wedge \mathbb{P} \subseteq \mathbb{F}$	Dynamic (see Section III-D)

graph:

$$SMT(HasRole(u,o)) = \bigvee_{(u,o')\in\mathcal{UA}} (o' = o \vee Reach(o', o))$$

where $Reach(o', o)$ represents the reachability from o' to o in the role inheritance graph, which can be further expanded into a disjunction of finite depth. Similarly, $HasTenantRole(u, o, t)$ is encoded as a reachability condition with tenant constraints. Since the SMT encoding of stub functions depends on the specific authorization graph instance, this item in Table I is marked as dynamically generated at runtime, indicating that its encoding is dynamically generated at runtime based on the actual role inheritance relationships. This design does not affect the completeness of verification, as all stub functions are expanded into equivalent logical constraints before verification.

For the verification efficiency problem with large-scale policy sets, we adopt the following optimization measures:

1) *CEGAR based on Horn Clauses:* The property checking problem is transformed into a Horn clause satisfiability problem, using CEGAR (CounterExample Guided Abstraction Refinement) [22] technology to gradually expand the policy scale in an abstraction-refinement loop. Specifically, in the initial phase, only an abstract subset of the policy set is verified; when a counterexample is found, the counterexample information is used to refine the abstraction and include more relevant policies in the verification scope, until verification passes or a genuine violation is found.
2) *Incremental SMT Solving:* Learned clauses and invariants are reused during the verification process to avoid redundant computation. When the policy set undergoes incremental updates (such as adding or deleting a small number of rules), only the changed parts need to be re-verified, rather than re-verifying the entire policy set.
3) *Policy Set Preprocessing:* Techniques such as redundant policy elimination and independent sub-policy set partitioning are used to preprocess and simplify the policy set, reducing the solving difficulty. For example, by using graph partitioning to split policies of different tenants into independent sub-problems, parallel verification can be performed, significantly reducing the total verification time.

Thanks to the expressiveness of SMT, we can also use it to characterize and verify complex policy properties. Below are some typical security properties in cloud computing scenarios:

- *Inter-tenant Permission Isolation:* Resources managed by different tenants cannot be accessed by each other; for example, users of tenant A cannot read or write files of tenant B. Formally, if $u \in \mathcal{U}_A \wedge v \in \mathcal{U}_B$, then $\forall pm \in \mathcal{PM}, \neg(u \xrightarrow{pm} v)$, i.e., there is no permitted permission transfer path from u to v.
- *Time and Geolocation-Based Access Restrictions:* Certain sensitive resources are only allowed to be accessed during working hours or at specific locations. For example, requiring $\forall r \in \mathcal{R}, (resource(r) = sensitive \to (time(r) \in [9{:}00, 18{:}00] \wedge location(r) \in \{office\}))$.
- *Data Flow-Related Policies:* Certain data can only be accessed in specific environments and cannot be propagated externally. For example, requiring "files marked as $secret$ can only be accessed by security team members $alice$ and bob, and cannot be sent outside the organization".

By encoding PML policies as SMT constraints, we can leverage modern constraint solving techniques to automatically analyze the semantic properties of policies, greatly improving the efficiency and reliability of policy design and management. It is worth noting that this formal method can also be combined with other formal methods (such as model checking, theorem proving, etc.) to achieve a more comprehensive and deeper understanding of policy behavior.

Table I lists the various syntactic structures of the PML language and their corresponding SMT encoding rules. Based on these encoding rules, we can convert any PML policy set Γ into an equivalent SMT formula $SMT(\Gamma)$, thereby enabling automated reasoning and verification using SMT solvers.

E. Policy Conflict Resolution

Writing security policies is one of the most challenging tasks in information security management. Policy errors can lead to serious consequences such as unauthorized access or data leakage. However, due to the complexity of security policies, even experienced developers find it difficult to write completely error-free policies. To address the policy conflict problem, this paper proposes a novel method based on Graph RAG (Graph-based Retrieval-Augmented Generation), as shown in Fig. 2. Graph RAG is a method that combines graph retrieval and generative question answering, which can fully leverage information in graph knowledge bases to discover potential problems in policies and propose modification suggestions.

First, we convert the PML policy set $\mathcal{P} = \{p_1, \ldots, p_n\}$ into an authorization graph $G(V, E)$ (following the construction

method in Section III-B). Each vertex $v \in V$ in the graph represents an entity (such as a user, role, or resource), and each edge $e \in E$ represents a relationship between entities (such as role assignment, role inheritance, permission assignment, etc.). Additionally, we introduce constraint edges in the graph to represent constraint relationships defined in domain axioms, such as mutual exclusion role constraints and cardinality constraints. These constraint edges do not participate in normal authorization decisions but are used for conflict detection. We define a policy-to-graph mapping function $\phi : \mathcal{P} \to G$: for a policy rule p_i, $\phi(p_i) = (V_i, E_i)$, where $V_i \subseteq V$ is the set of entity vertices extracted from p_i (subjects, objects, actions, etc.), and $E_i \subseteq E$ is the set of directed edges created based on the semantics of p_i (allow/deny) and the role assignments or inheritance relationships involved.

Then, we store the graph G into a Graph RAG knowledge base $\mathcal{K}$. Graph RAG consists of two parts: the retriever $\mathcal{R}$ and the generator $\mathcal{G}$. Given a new policy p_i, the retriever $\mathcal{R}$ first retrieves from the knowledge base $\mathcal{K}$ the subgraphs $\{g_1, \ldots, g_k\}$ that are similar or related to $\phi(p_i)$, where each $g_j \subseteq G$.

The retriever $\mathcal{R}$ computes similarity based on the structural features of the graph (such as degree distribution, connectivity, etc.) and semantic features (such as embedding representations of entities and relationships). Specifically, we use Llama-3 [23] (specifically the Llama-3-70B-Instruct variant) to encode the textualized representations of $\phi(p_i)$ and g_j, obtaining their embedding vectors $\vec{u}_i$ and $\vec{v}_j$, and then use the cosine similarity between them to measure their proximity in the latent semantic space. Graph RAG retains the top-k ($k = 5$) most similar subgraphs $\{g_1, \ldots, g_k\}$ with cosine similarity above a threshold $\tau = 0.7$ for the next step. This threshold was selected through grid search on a validation set containing 500 annotated conflict instances.

Next, Graph RAG analyzes potential conflicts or inconsistencies between the retrieved subgraphs and $\phi(p_i)$. We define a conflict detection function $\mathcal{C}(\phi(p_i), g_j)$: it returns 1 if there is a conflict between $\phi(p_i)$ and g_j, and 0 otherwise. This function makes its determination based on the following two categories of specific constraint conditions:

Structural Constraints:

1) *Policy Cycle Detection:* After adding the edges of $\phi(p_i)$ into the subgraph g_j, check whether a directed cycle is produced in the role inheritance subgraph. If a cycle $o_1 \to o_2 \to \cdots \to o_1$ exists, it is determined to be a cyclic inheritance conflict.
2) *Mutual Exclusion Role Detection:* Check whether $\phi(p_i)$ assigns the same user to two roles defined as mutually exclusive by domain axiom Ξ.
3) *Cardinality Constraint Detection:* Check whether role assignments exceed the predefined maximum user count limit.

Semantic Constraints:

1) *Permission Redundancy Detection:* If the permissions granted by $\phi(p_i)$ are already covered by an existing policy in g_j (i.e., the new policy is a subset of an existing policy), it is determined to be redundant.
2) *Permission Conflict Detection:* If the effect (allow/deny) of $\phi(p_i)$ contradicts an existing policy in g_j for the same subject-object-action triple, it is determined to be a conflict.
3) *Least Privilege Violation Detection:* If the scope of permissions granted by $\phi(p_i)$ is too broad (e.g., using wildcards to match all resources) and the involved roles are not in the predefined set of privileged roles, it is determined to be a violation of the principle of least privilege.

If a conflict is detected, i.e., there exists some g_j such that $\mathcal{C}(\phi(p_i), g_j) = 1$, Graph RAG generates a natural language question q describing the discovered conflict and asking the user how to handle it. The generator $\mathcal{G}$ takes $\phi(p_i)$ and g_j as input and produces the question q:

$$\mathcal{G}(\phi(p_i), g_j) = q \tag{7}$$

$\mathcal{G}$ employs a pretrained generative language model as its backbone, taking the structured representation of the graph as input and producing a natural language question as output. Specifically, we use OPT-1.3B [24] as the base model and adopt LoRA [25] (rank $r = 16$) for parameter-efficient fine-tuning, training for 5 epochs on 3,200 conflict examples (learning rate 2×10^{-4}, batch size 16), which takes approximately 2.1 hours on a single NVIDIA A100 (40GB). The model input linearizes the retrieved subgraph g_j and $\phi(p_i)$ into a sequence of `(source, relation, target)` triples, concatenated in the order "entity–relation–constraint", so as to preserve the graph topology while adapting to serialized input. The average latency of a single question generation is approximately 0.4 seconds (about 320 input tokens and 90 output tokens), and each policy is resolved in an average of 1.3 iterations. During the training phase, we constructed a training set containing 3,200 conflict examples, covering all six conflict types described above (three each for structural constraints and semantic constraints). Each conflict type corresponds to several question templates, which contain placeholders (such as role names, resource names, etc.) that are filled with actual policy elements during training. The training data comes from three sources: manually annotated real conflict cases from the ibm-loopback and ms-monitor datasets (approximately 40%), synthesized policy pairs based on conflict patterns (approximately 40%), and actual reports collected from issue trackers of open-source authorization systems (approximately 20%). Among them, structural conflicts (cyclic inheritance, mutually exclusive roles, cardinality constraints) have clear formal decision criteria, and their labels are automatically and unambiguously determined by structural constraint checks. Semantic conflicts (permission redundancy, permission conflict, least privilege violation) are inherently context-dependent and may not have a unique standard answer; for this reason, we use a pre-agreed organization-level security baseline (the minimum necessary permissions for each role and the access scope of sensitive resources) as a reference, with domain experts independently annotating, and divergent samples being determined by adjudication of a third expert. During inference, given a new conflict instance $(\phi(p_i), g_j)$, the model can automatically generate the corresponding question.

Below is an example prompt for conflict detection and question generation:

```
 The user submitted the following new policies:
p, anonymous_user, data_server, read, id in
  ↪ [1, 1000]
p, anonymous_user, data_server, write, id in
  ↪ [1, 1000]
p, admin, data_server, read, id >= 1
p, admin, data_server, write, id >= 1
p, admin, admin_portal, *, *
By comparing with existing policies, the
  ↪ following issues were found:
1. anonymous_user is granted write permission
  ↪ on data_server for ids in the range [1,
  ↪ 1000], which conflicts with the principle
  ↪ of least privilege.
2. admin is granted read and write permissions
  ↪ on all data in data_server, which may
  ↪ cause privilege escalation.
Please propose reasonable modification
  ↪ suggestions and generate natural language
  ↪ questions asking the user how to handle
  ↪ this:
```

$\mathcal{G}$, based on the above prompt and combining the new policy p_i with the similar subgraph g_j, generates the following question q:

```
I found the following issues with the newly
  ↪ submitted policies:
1. Granting write permission on data_server to the
  ↪ anonymous_user role may violate the
  ↪ principle of least privilege, creating a
  ↪ risk of data being illegally tampered
  ↪ with. It is recommended to grant only the
  ↪ necessary read permissions.
2. Granting read and write permissions on all
  ↪ data in data_server to the admin role may
  ↪ lead to sensitive data leakage. It is
  ↪ recommended to restrict the data range
  ↪ accessible to admin based on actual needs.
Do you agree with the above modification
  ↪ suggestions? Or do you have otherw
  ↪ considerations?
```

The user provides feedback a based on the question q, indicating how to handle the conflict (such as retaining the original policy, adopting the new policy, or making modifications). Graph RAG generates a policy modification Δp_i based on the feedback:

$$\Delta p_i = \mathcal{G}(a, \phi(p_i), g_j) \tag{8}$$

Iterative Termination Mechanism: To ensure the convergence and practicality of the conflict resolution process, we introduce the following termination mechanisms: (1) *Maximum Iteration Limit:* The upper limit of conflict resolution iterations per policy is set to $N_{\max} = 3$; when this limit is exceeded, the system reports the remaining unresolved conflicts to the user and terminates the iteration; (2) *Monotonic Conflict Decrease Constraint:* After each iteration, the system verifies whether the current number of conflicts is strictly less than the number in the previous iteration, i.e., $|\mathcal{C}^{(t+1)}| < |\mathcal{C}^{(t)}|$; if this condition is not met, the iteration terminates and the user is prompted for manual intervention; (3) *Modification Scope Limitation:* Each generated policy modification Δp_i is only allowed to modify policy rules directly related to the current conflict; modifying unrelated policies is not allowed, thereby avoiding the introduction of new conflicts. In experiments, we observed that over 95% of conflicts are resolved within 2 iterations, with an average iteration count of 1.3.

After generating the modification, Graph RAG performs conflict detection on Δp_i again to prevent introducing new inconsistencies. If confirmed to be correct, the modification Δp_i is applied to the policy p_i and the authorization graph G, and the knowledge base $\mathcal{K}$ is updated:

$$p_i^* = p_i + \Delta p_i \tag{9}$$

$$G^* = G + \phi(\Delta p_i) \tag{10}$$

$$\mathcal{K}^* = \mathcal{K} \cup \{G^*\} \tag{11}$$

where p_i^* denotes the modified policy, G^* denotes the updated authorization graph, and $\mathcal{K}^*$ denotes the new knowledge base.

Algorithm 2 presents the main flow of Graph RAG, including the above termination mechanisms. The main steps of this algorithm are as follows: first, convert the policy set $\mathcal{P}$ into the authorization graph G and store it in the knowledge base $\mathcal{K}$ (lines 1-5). Then iterate over each policy p_i and retrieve the related subgraphs $\{g_1, \ldots, g_k\}$ (lines 6-7). For each subgraph g_j, detect whether there is a conflict with $\phi(p_i)$ (line 9). If a conflict exists, generate a question q asking the user how to handle it (lines 10-11). Generate a policy modification Δp_i based on user feedback and apply it to p_i and G (lines 12-15). When the iteration count exceeds $N_{\max}$ or the number of conflicts does not decrease, terminate the conflict resolution for the current policy (lines 16-18). Finally, store the updated authorization graph in the knowledge base and return the modified policy set (lines 21-23).

IV. Evaluation

A. Case Study

LoopBack [26] is a popular open-source web framework launched by IBM for building APIs and microservices. It provides a powerful set of tools and libraries that help developers quickly create REST APIs. Suppose there are three roles in the LoopBack system: regular user (`user`), auditor (`auditor`), and administrator (`admin`). System resources are grouped by business line (biz1, biz2), and resources of different business lines are located under different paths (such as `/biz1/models`, `/biz2/datasources`). The system requires that regular users can only access resources related to their own business line, and only have read permissions; auditors can read audit logs (`/audit_log`) of all business lines; administrators can read and write all resources; the permissions of auditors and administrators are mutually exclusive and cannot be held simultaneously. We can use the following PML model and policies to implement the above requirements:

Algorithm 2: Policy Conflict Resolution With Termination Guarantees.

Require: Policy set $\mathcal{P}$, knowledge base $\mathcal{K}$, retriever $\mathcal{R}$, generator $\mathcal{G}$, max iterations $N_{\max}$
Ensure: Updated policy set $\mathcal{P}^*$, knowledge base $\mathcal{K}^*$

$G \leftarrow \emptyset$ ▷ Initialize authorization graph
for $p_i \in \mathcal{P}$ **do**
  $G \leftarrow G \cup \phi(p_i)$ ▷ Add policy to graph
end for
$\mathcal{K} \leftarrow \mathcal{K} \cup \{G\}$ ▷ Add graph to knowledge base
for $p_i \in \mathcal{P}$ **do**
  $\{g_1, \ldots, g_k\} \leftarrow \mathcal{R}(\phi(p_i), \mathcal{K})$ ▷ Retrieve relevant knowledge
  $iter \leftarrow 0$, $prev\_conflicts \leftarrow \infty$ ▷ Initialize iteration control
  for $g_j \in \{g_1, \ldots, g_k\}$ **do**
    if $\mathcal{C}(\phi(p_i), g_j) = 1$ **then** ▷ Check for conflicts
      $q \leftarrow \mathcal{G}(\phi(p_i), g_j)$ ▷ Generate query
      $a \leftarrow$ GET_USER_FEEDBACK(q) ▷ Get user feedback
      $\Delta p_i \leftarrow \mathcal{G}(a, \phi(p_i), g_j)$ ▷ Generate policy update
      $p_i \leftarrow p_i + \Delta p_i$ ▷ Update policy
      $G \leftarrow G + \phi(\Delta p_i)$ ▷ Update graph
      $iter \leftarrow iter + 1$
      $cur\_conflicts \leftarrow |\{g \in \{g_1, \ldots, g_k\} : \mathcal{C}(\phi(p_i), g) = 1\}|$
      if $iter \geq N_{\max}$ **or** $cur\_conflicts \geq prev\_conflicts$ **then**
        break ▷ Terminate: max iterations or no progress
      end if
      $prev\_conflicts \leftarrow cur\_conflicts$
    end if
  end for
end for
$\mathcal{K}^* \leftarrow \mathcal{K} \cup \{G\}$ ▷ Update knowledge base
$\mathcal{P}^* \leftarrow \{p_1, \ldots, p_n\}$ ▷ Update policy set
return $\mathcal{P}^*, \mathcal{K}^*$

PML model:

```
  r = sub, obj, act
  p = sub, obj, act, cond
  g = _, _
  g2 = _, _, _
  e = some(where (p.eft == allow))
  m = g(r.sub, p.sub) && key-
Match2(r.obj, p.obj) && regex-
Match(r.act, p.act) && eval(p.cond)
```

PML policy rules:

```
  p, user, /biz{1}/*, GET, eval(env.user.
biz == regexp.split(env.obj,'/')[1])
  p, auditor, /audit_log, GET,
  p, admin, /**, (GET)|(POST)|(DELETE)|
(PATCH),
  g, auditor_group, auditor
  g, admin_group, admin
  g2, auditor_group, ad-
min_group, eval(env.user.roles == 'au-
ditor' || env.user.roles == 'admin')
```

In the PML model definition, the `g(r.sub, p.sub)` call in the matcher invokes the stub function `g` to check whether the request subject inherits the role specified in the policy (i.e., triggering the *HasRole* semantics described in Section III-A); the `cond` field introduces conditional expressions in the policy, enabling dynamic authorization based on the request context. `p.eft == allow` indicates that the policy effect is "allow", which is one of the conditions for successful authorization. In the matcher definition, we use `keyMatch2` to support parameter matching in paths, e.g., `/biz1/*` can match specific paths such as `/biz1/models`. `eval` is used to execute custom conditional expressions for authorization based on the request context. Compared with Cedar's static attribute-based mapping, PML can perform authorization more flexibly based on the request context without frequently modifying user attributes.

PML's mutually exclusive roles can be expressed using `g2` rules. The form of a `g2` rule is `g2, role1, role2, cond`, indicating that `role1` and `role2` have a mutual exclusion constraint when condition `cond` is satisfied, i.e., the same subject cannot be assigned both roles simultaneously. In the above `g2` rule, we specify that `auditor_group` and `admin_group` are mutually exclusive when the user holds both `auditor` and `admin` roles. The mutual exclusion constraint takes effect as an implicit deny rule during policy evaluation, i.e., when the request subject matches both mutually exclusive roles, Casbin will deny the request. This ensures that users cannot hold the permissions of both roles simultaneously. Compared with OpenFGA, PML can easily express such advanced role constraints without using additional APIs or query statements.

It is worth emphasizing that the above case also reflects PML's support for ABAC and multi-tenancy, and demonstrates how they are represented in the authorization graph. In terms of ABAC, the `eval(env.user.biz == regexp.split(env.obj,'/')[1])` in the first policy is a typical attribute-based dynamic condition: it dynamically determines access permissions based on the user attribute `biz` in the request context and the resource path. In the authorization graph, this condition does not change the vertex set of the graph but exists as a predicate label on the directed edge from `user` to the corresponding resource permission vertex—only when the request context makes the predicate true does this edge take effect in the reachability search. In terms of multi-tenancy, we can treat different business lines `biz1` and `biz2` as different tenants, and we only need to attach a tenant identifier to the policy rules and extend the model accordingly:

TABLE II
EXPRESSIVENESS OF DIFFERENT AUTHORIZATION METHODS

Authorization method	Supported feature											
	ACL	RBAC	ABAC	SoMP[1]	MT[2]	RESTful	Graph	SMT	DC[3]	SF[4]	RH[5]	CP[6]
OpenFGA [7]	✓	✓	✓		○	✓			✓	✓	✓	✓
SpiceDB [8]	✓	✓	✓[7]			✓	✓[8]		✓	✓	✓	✓
Cedar [6]	✓	✓	✓	✓		✓		✓	✓	✓	✓	✓
Rego [9]	✓	✓	✓		○	✓				✓		✓
XACML [10]	✓	✓	✓		✓	✓			✓	✓		✓
Ponder [15]		✓	✓							✓	✓	✓
Casbin	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓

[1] Separation of model and policy (SoMP): separates the description for the access control model from policy rules.
[2] Multi-tenancy (MT): allows a cloud-based system to support isolated permissions for multiple tenants.
[3] Decision combination (DC): can form a single decision from all the policies that apply to an access request.
[4] Stub function (SF): if external logic is supported.
[5] Role hierarchy (RH): supports the RBAC role hierarchy or not.
[6] Contextual Policies (CP): supports policies that take context into account.
[7] SpiceDB supports ABAC-like functionality through its caveats mechanism.
[8] SpiceDB uses implicit graph traversal for authorization; it does not expose an explicit graph model API.
○ Partial multi-tenancy support realized through application-level workarounds (e.g., embedding tenant identifiers into relation tuples or rules) rather than a native mechanism.

```
  p, user, /biz{1}/*, GET, eval(env.user.
tenant == 'biz1' && env.user.biz == reg-
exp.split(env.obj,'/')[1])
  p, user, /biz{2}/*, GET, eval(env.user.
tenant == 'biz2' && env.user.biz == reg-
exp.split(env.obj,'/')[1])
```

In this case, Casbin partitions the authorization graph into two mutually disconnected subgraphs `biz1` and `biz2` based on the tenant identifier `tenant`, and a request from the `biz1` tenant only performs reachability search on the `biz1` subgraph, thereby ensuring permission isolation between tenants at the graph structure level.

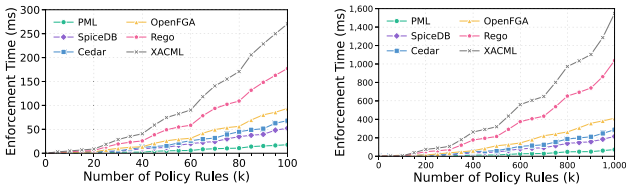
B. Expressiveness Analysis

We compared the similarities and differences in functional features between Casbin and other mainstream authorization frameworks and languages, as shown in Table II. It can be seen that Casbin has comprehensive capabilities in terms of supported authorization models, semantic expression, and formal verification. In contrast, traditional authorization frameworks such as OPA and XACML, although relatively complete in basic access control models, lack capabilities in advanced features such as graph modeling, SMT verification, and LLM assistance, making it difficult to cope with the complex and changing authorization requirements in cloud environments. Although SpiceDB implicitly uses graph traversal for authorization decisions, it does not provide an explicit graph model interface, nor does it support features such as SMT verification and the separation of model and policy. Emerging authorization frameworks such as OpenFGA and Cedar designed for cloud environments, although enhanced in multi-tenancy and REST interfaces, lack support for graph semantic expression and policy conflict resolution, and have limited expressiveness in handling complex policies such as cross-domain authorization and role mutual exclusion. It should be noted that although systems such as OpenFGA and Rego do not natively provide a multi-tenancy mechanism, they can achieve approximate tenant isolation through application-level workarounds such as explicitly embedding tenant identifiers into relation tuples or rules; therefore, in Table II we annotate their multi-tenancy support as partial support (○) rather than completely unsupported.

C. Performance

This section compares the performance of different authorization methods in real scenarios. We selected two representative applications: ibm-loopback and ms-monitor. The experiments were conducted on an Ubuntu 20.04 server equipped with an Intel Xeon 8369B CPU (2.7 GHz, 4 cores) and 32 GB of memory, ensuring consistency in hardware and software environments. Casbin was compiled based on Golang 1.23, SpiceDB was v1.38.1, OpenFGA was v1.8.4, Cedar was v4.3.1, Rego was v1.0.1, and XACML was v3.0. To generate realistic, complex, and diverse test policies, we analyzed the permission policies provided by the official use cases of each system and extracted common patterns and structures. Based on these templates, we developed a policy generation tool that can automatically generate scenario-specific policy sets based on parameters such as the number of policies, the number of subjects and objects, and the depth of role hierarchy.

To ensure the fairness of the comparison, we designed a set of semantically equivalent authorization requirements for each scenario and performed equivalent encodings for each system's policy model: RBAC role inheritance is encoded as relation tuples in OpenFGA/SpiceDB, as role entities and the `in` relation in Cedar, and as equivalent rule sets in Rego/XACML. ABAC attribute conditions are encoded as `when` clauses in Cedar, as caveats in SpiceDB, and as attribute comparison expressions in Rego. Multi-tenancy isolation is implemented through each system's namespace, policy partition, or tenant prefix respectively. We verified the semantic consistency of the different encodings by executing each system on the same request test set and comparing their authorization decisions one by one (the decision consistency rate on the test request set reached 100%). In addition, all systems were run under their officially recommended



(a) ibm-loopback (b) ms-monitor

Fig. 3. Performance comparison on enforcement latency.

optimization configurations (such as enabling caching, indexing, and incremental loading) to avoid unfair comparisons caused by configuration differences. It is worth noting that the above test set is not limited to RBAC/ReBAC-style workloads but also includes ABAC rules with wildcards and attribute conditions, as well as context-aware rules that depend on the request context (such as access time, business line, and user attributes), thereby enabling a more comprehensive evaluation of the performance of each method in dynamic, context-dependent authorization scenarios.

For the ibm-loopback scenario, we simulated 10 business lines and 100 resource types, and randomly generated test sets composed of 1,000 to 100,000 policies, covering common models such as RBAC and ABAC, and rule structures such as multi-level role inheritance and resource paths with wildcards. The experiments gradually increased the number of policies, repeated the test 20 times, and recorded the average response latency and standard deviation of each permission check request. Fig. 3(a) shows the performance of each method under different policy scales. It can be seen that the average policy evaluation time of Casbin grows the slowest, reaching 17.8± 1.2 ms at 100,000 policies, outperforming SpiceDB's 52.3± 4.8 ms, Cedar's 68.1± 6.3 ms, OpenFGA's 93.7± 7.6 ms, Rego's 177.3± 12.8 ms, and XACML's 270.5±18.6 ms. SpiceDB's performance is better than OpenFGA and Cedar, thanks to its underlying graph traversal optimization, but still inferior to Casbin's explicit graph model optimization. Casbin also maintains a 3-5x performance advantage at other data points.

For the ms-monitor scenario, we generated an ultra-large-scale dataset containing 1,000 tenants, 2,000 services, and 20,000 APIs, with the total number of policies increasing from 100,000 to 1,000,000. As shown in Fig. 3(b), Casbin's performance again leads other methods. At 1 million policies, Casbin's average request latency is 73± 4.1 ms, SpiceDB's is 215± 14.2 ms, Cedar's is 288± 17.6 ms, OpenFGA's is 412±29.3 ms, Rego's exceeds 1 s, and XACML's reaches 1.5 seconds. SpiceDB also shows performance between Casbin and Cedar in this scenario, with its graph traversal mechanism having certain advantages in handling multi-tenant scenarios, but due to the lack of explicit graph partitioning and graph compression optimizations, it still has a significant gap with Casbin on ultra-large-scale policy sets. More importantly, Casbin's performance grows relatively linearly with the policy scale, without dramatic deterioration, demonstrating its good scalability.

D. Ablation Study

We conducted a series of ablation experiments on three datasets: ibm-loopback, intel-rmd, and ms-monitor, to gain a deeper understanding of the role of each module in Casbin. We examined the effects of graph model optimization (GMP), SMT verification (SMT), and interpretability modules. Graph model optimization includes three sub-modules: graph partitioning (GP), graph compression (GC), and graph indexing (GI). In terms of interpretability, we compared variants using only the language model (LLM), using a regular RAG knowledge base (RAG), using Graph RAG (GRAG), and not using LLM at all. Table III shows that the complete Casbin model achieves the best performance on all metrics. Removing graph optimization significantly increases latency and reduces throughput, but has little impact on correctness and authoring time. Completely removing the graph model further deteriorates performance and accuracy. SMT verification has a significant impact on correctness, with a 6.9 percentage point drop in correctness after removal. In terms of interpretability, the correctness of using LLM alone is very low, improves with the addition of RAG, but is still significantly lower than GRAG. GRAG, by constructing graphical knowledge representations, can better capture the relationships between policy elements, thereby significantly improving correctness and authoring efficiency. Completely removing the LLM further reduces correctness to 62.7%, and authoring time also increases significantly, highlighting the important value of large models in policy generation. Specifically, for different types of constraints and conflicts, the performance of each variant varies. The GRAG variant achieves a correctness of 93.1% in handling structural conflicts (such as cyclic inheritance and mutually exclusive roles) and 85.7% in handling semantic conflicts (such as permission redundancy and least privilege violations). In contrast, the RAG-only variant has correctness of 79.2% and 73.8% on structural and semantic conflicts respectively, while the LLM-only variant has 71.5% and 64.7% respectively. This indicates that Graph RAG's graph-structured knowledge representation has significant advantages in capturing the topological relationships between policy elements, especially in handling conflict types that require understanding graph structures.

E. Formal Verification Analysis

We use the latest Z3 SMT solver (v4.14.4) and its Python API to evaluate Casbin's formal verification capability. The verification experiments were conducted on a workstation equipped with an Intel Xeon 8369B CPU (2.7 GHz, 4 cores) and 32 GB of memory, with Ubuntu 20.04 as the operating system. We set Z3 to use incremental mode and enable auto-configuration to obtain the best performance. The experimental datasets are the aforementioned ibm-loopback and ms-monitor, containing 1,000,000 RBAC and ABAC policies. The datasets have undergone necessary anonymization and randomization through the aforementioned policy generation tool, thereby preserving the structural characteristics of real policies while avoiding the leakage of sensitive information. Policy types cover various scenarios such as RBAC, ABAC, and multi-tenancy. To objectively evaluate the detection capability of formal verification, we used the following method to construct the verification benchmark

TABLE III
RESULTS OF THE ABLATION STUDY

Variant	Optimization			SMT	Interpretability			Enforcement time (ms)	Throughput (req/s)	Correctness (%)	Policy writing time (s)
	GP	GC	GI		Graph RAG	RAG	LLM				
w/o GP		✓	✓	✓	✓		✓	28.4	3,521	-	-
w/o GC	✓		✓	✓	✓		✓	41.5	2,409	-	-
w/o GI	✓	✓		✓	✓		✓	57.2	1,748	-	-
w/o GMP				✓	✓		✓	98.6	1,014	-	-
w/o SMT	✓	✓	✓		✓		✓	-	-	82.3	-
No-LLM	✓	✓	✓	✓				-	-	62.7	311.5
LLM-only	✓	✓	✓	✓			✓	-	-	68.1	292.6
RAG-only	✓	✓	✓	✓		✓		-	-	76.5	106.3
Graph RAG	✓	✓	✓	✓	✓		✓	17.8	5,618	89.2	27.9

GP: Graph Partitioning, GC: Graph Compression, GI: Graph Indexing, GMP: Graph Model Optimization

TABLE IV
FORMAL VERIFICATION RESULTS

Defect type	Prop. (%)	Rec. (%)	Prec. (%)
Inconsistency	28.2	97.8	96.2
Incompleteness	21.7	95.4	94.1
Mutual Exclusion Conflict	18.9	98.1	97.6
Cyclic Inheritance	14.3	96.9	95.5
Constraint Violation	7.5	93.2	90.8
Cardinality Constraints	3.2	91.5	89.3
Others	6.2	90.7	88.4

Prop.: Proportion, Rec.: Recall, Prec.: Precision

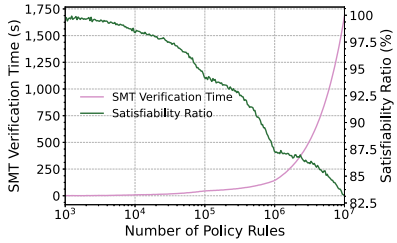


Fig. 4. SMT verification time and satisfiability ratio.

(ground truth): First, we manually injected violations of known types and quantities (including consistency conflicts, mutual exclusion conflicts, cyclic inheritance, and seven other types) into the original policy set, with an injection ratio of approximately 5% of the total policies. Then, three domain experts with more than 5 years of security policy experience independently reviewed the injected violations to ensure the correctness of their type annotations. The expert review targeted two parts: the first is all manually injected violations (whose locations and types are known in advance); the second is all alarms reported by the SMT solver, as well as approximately 5,000 policy samples obtained by stratified sampling from the policies judged as compliant (used to estimate the false negative rate). The final ground truth is jointly constituted by "known injected violations + expert-confirmed SMT alarms + unintended violations found by experts in the sampled samples". Based on this benchmark, we compared the detection results of the SMT solver with the expert annotations to calculate recall and precision. False positives are defined as cases where SMT reports a violation but experts judge it as compliant; false negatives are defined as cases where experts annotate a violation but SMT does not detect it.

The results show that even at the scale of 1 million policies, Casbin's SMT verification still maintains a recall of more than 92.1% and precision of more than 94.5%, with average detection time controlled within 143.2 seconds. Table IV further shows the specific detection results for different violation types. It can be seen that for common violation types such as Mutual Exclusion Conflict and Cyclic Inheritance, Casbin's recall and precision are both above 95%. The high detection rates of mutual exclusion conflicts and cyclic inheritance benefit from their explicit structural constraints (such as the SMT encoding of mutually exclusive roles and acyclic constraints defined in Section III-D), enabling the SMT solver to make precise judgments. The detection rates of Constraint Violation and Cardinality Constraints are relatively lower, mainly because these violation types involve more complex numerical reasoning and quantified constraints, which require higher reasoning capabilities from the SMT solver. False positives mainly occur in cases where there are subtle semantic differences between policy rules but no real conflict (for example, two policies respectively allowing and denying access to the same resource but acting on different attribute conditions, where the SMT solver in some cases cannot fully distinguish the mutual exclusivity of attribute conditions). From the perspective of policy model dimensions, the overall recall (95.3%) and precision (96.1%) of pure RBAC policies are higher than those of ABAC policies (recall 89.7%, precision 91.2%), mainly because ABAC policies involve more attribute condition combinations, increasing the reasoning complexity of the SMT solver.

Fig. 4 shows the curves of SMT verification time and satisfiability ratio as a function of policy scale. It can be seen that at the scale of 1 million policies, the average SMT verification time is 143.2 seconds, and the satisfiability ratio is 87.3%. As the number of policies increases from 1,000 to 1 million, the SMT verification time grows from 0.7 seconds to 143.2 seconds, basically showing a linear trend. The satisfiability ratio gradually decreases from close to 100% to 87.3%. It should be noted that a lower satisfiability ratio does not mean verification failure, but reflects the increasing probability of violations and conflicts in the policy set as the policy scale and complexity grow. A satisfiability ratio of 87.3% means that among 1 million policies, approximately 12.7% of verification queries discovered policy violations, which is basically consistent with our manually injected violation ratio of approximately 5% and the additional conflict ratio introduced by the policy generation tool during large-scale random generation. In other words, the satisfiability

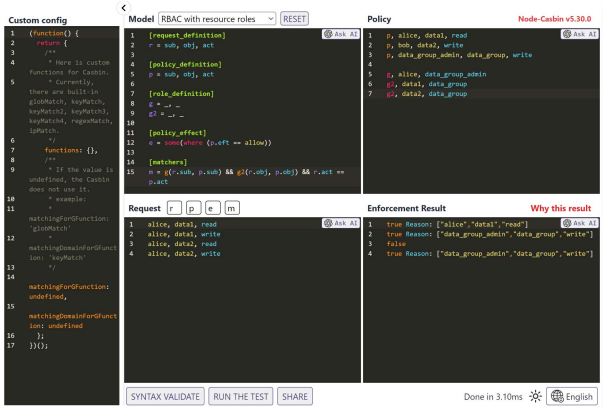


Fig. 5. The prototype of PML editor.

ratio of SMT verification is not a metric for measuring the correctness of the solver itself, but reflects the density of potential violations in the policy set to be verified. To further verify this, we repeated the experiment on a clean policy set that had been manually reviewed and contained no injected violations, and observed that the satisfiability ratio remained stable above 99%, indicating that the SMT solver itself is highly reliable in judging compliant policies. The above results jointly demonstrate that Casbin's SMT verification can maintain high detection accuracy and processing capability on large-scale complex policy sets. This benefits from the multiple optimization measures we proposed in Section III-D, such as Horn-clause-based CEGAR, incremental SMT solving, and policy simplification.

Regarding the scalability of larger policy sets, we further conducted actual verification experiments at the scale of 10 million policies. The measured results show that the average SMT verification time is 1,683 seconds, and the verification time exhibits a slight super-linear growth with the policy scale. Further analysis indicates that when the policy set contains a large number of nested quantified constraints or complex role inheritance hierarchies (with depth exceeding 10 levels), the SMT solver may face rapid expansion of the search space. In our experiments, the worst-case verification time was approximately 3.2 times the average time, occurring in the ms-monitor dataset containing deep role inheritance and a large number of mutual exclusion constraints. For such extreme scenarios, this can be mitigated by adding policy set preprocessing (such as limiting the depth of role inheritance) or adopting approximate verification methods. It should be noted that the time consumption of the above complete verification is applicable to offline analysis scenarios; for online scenarios where policies are continuously changing, Casbin adopts the incremental SMT solving method described above, only re-verifying the affected sub-policy set rather than the entire policy set, thereby significantly reducing the verification overhead per update.

F. Policy Interpretability

We designed a prototype system for the policy conflict resolution method based on CodeMirror, with its user interface shown in Fig. 5. To evaluate the interpretability of Casbin policies, we conducted a user study. The study involved 120 participants with different experience levels, including 52 (43.3%) junior developers with 1-3 years of security policy authoring experience, 36 (30%) intermediate developers with 3-5 years of related experience, and 32 (26.7%) senior developers with more than 5 years of experience. The participants' experience levels were assessed through a pre-study questionnaire, which included the types of policy languages they had used, the number of authorization system projects they had participated in, and their self-assessed proficiency. We randomly assigned the participants to four language groups (PML, Cedar, OpenFGA, XACML), with 30 people in each group, and ensured that each group had a balanced distribution of experience levels (each group containing approximately 13 junior, 9 intermediate, and 8 senior developers). Each participant was asked to use their assigned policy language to complete three policy authoring tasks in real scenarios: (1) the user role and permission management policy of Intel's RMD (Resource Management Daemon) system; (2) the RBAC policy in IBM's LoopBack framework; (3) a fine-grained ABAC permission policy for a Microsoft internal monitoring service. These tasks cover various authorization models from basic ACL to complex RBAC and ABAC. In terms of task format, each participant was required to write complete policies from scratch: we provided them with unified input materials, including access control requirements described in natural language, descriptions of the target system's architecture and resource structure, and several example requests with their expected authorization results. The participant's output was a complete policy file (containing model definitions and policy rules) that could be directly loaded and executed. In terms of correctness evaluation, considering that the same requirement may have multiple semantically equivalent policy formulations, the experts did not perform character-by-character comparison but instead replayed the submitted policies based on the aforementioned example request set (covering typical allow and deny cases), using the proportion of requests whose authorization decisions are consistent with the reference answers as the correctness rate. As long as the submitted policy is semantically equivalent to the reference answer, it is judged as correct even if the syntactic form is different. To control for learning effects, we provided all subjects with a unified 60-minute training session. The training content was consistent across all groups: the first 20 minutes explained the general concepts of access control (RBAC, ABAC, etc.), the middle 30 minutes provided syntax explanations and example exercises for the policy language assigned to each group (each group used examples of the same difficulty but in different languages), and the last 10 minutes were free practice time. The training materials were jointly written by two teaching experts familiar with all four languages, ensuring that the materials of each group were comparable in coverage depth and example complexity. Policy editing used CodeMirror v6 as the editor, and the editors of each language group were configured with corresponding syntax highlighting and basic auto-completion functions. To ensure the objectivity of the evaluation, we invited domain experts in advance to use various policy languages to formulate standard answers containing dozens to hundreds of policies for each scenario, as references for evaluating the policies submitted by the participants.

TABLE V
RESULTS OF THE POLICY WRITING USER STUDY

Language	intel-rmd			ibm-loopback			ms-monitor		
	Time (s)	Comp. (%)	Corr. (%)	Time (s)	Comp. (%)	Corr. (%)	Time (s)	Comp. (%)	Corr. (%)
Cedar	1,895	88.5	77.3	1,513	91.2	73.5	2,047	82.6	71.8
OpenFGA	2,304	81.7	72.6	1,972	85.4	65.3	2,631	78.1	68.2
XACML	2,571	85.3	74.9	2,185	88.9	70.1	2,814	75.5	66.7
PML	854**	92.8*	89.2**	672**	97.3**	85.7**	1,203*	87.2*	82.9*

** $p < 0.01$, * $p < 0.05$ compared with other groups in the same task.
Comp.: Completion, Corr.: Correctness

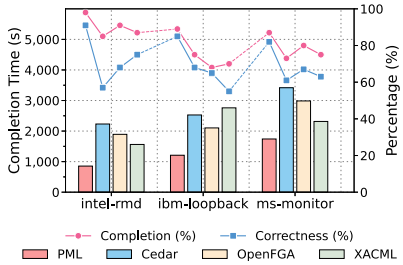


Fig. 6. Comparison of task completion time and correctness.

The experimental results are shown in Table V. Completion time represents the average time required for participants to complete the tasks. Completion rate and correctness rate respectively represent the average coverage and correctness of the policies submitted by participants relative to the reference answers, given by expert review. Statistical significance was assessed using independent-samples t-tests, with $p < 0.05$ as the significance level. It can be seen that the completion time of the PML group is significantly shorter than that of other groups across all three tasks ($p < 0.01$). Taking Intel RMD as an example, the PML group only needed an average of 854 seconds, less than half of the Cedar group, and far below the OpenFGA and XACML groups. In terms of correctness, the correctness rate of the PML group is above 80%, significantly higher than other languages by 10-30 percentage points ($p < 0.05$). This is mainly due to PML's efficient policy matching mechanism based on the graph model and the intelligent policy editor driven by Graph RAG. The editor leverages the graph knowledge base to provide users with real-time error correction and optimization suggestions, greatly reducing the threshold for policy authoring. In contrast, OpenFGA's correctness rate is lower, e.g., only 65.3% in the ibm-loopback task. This may be due to the high learning cost of its declarative syntax. As the task complexity increases, the completion rate and correctness rate of each group decline, but the PML group has the smallest decline. For example, in the ms-monitor task, the completion rate of the PML group is 87.2%, and the correctness rate is 82.9%, while other groups are on average 20-30 percentage points lower. This indicates that PML is more robust in handling complex policies. In contrast, XACML's completion rate in this task is only 75.5%, possibly because its XML-based syntax is verbose when expressing complex logic. Finally, 86.7% of participants believed that PML significantly improved authoring efficiency and reduced the difficulty of policy authoring. Fig. 6 further shows the average task completion time and correctness rate of each group. From the figure, it can be intuitively seen that the PML group has the shortest completion time on all tasks, and as the task complexity increases, the advantage of PML becomes more apparent.

V. DISCUSSION

A. Limitations

Currently, the proposed method has the following limitations.

Execution performance: Although Casbin's execution performance at the scale of millions of policies is already better than existing methods, it may still be unsatisfactory for some applications with extremely high real-time requirements (such as high-frequency securities trading, which requires sub-millisecond latency). The current average latency at the scale of 1 million policies is 73 ms, which is still far from the requirements of such extreme scenarios. In the future, further performance optimization is needed at the algorithm and architecture level, such as introducing GPU-accelerated sparse matrix operations and optimizing the caching strategy of graph indexes.

Scalability of formal verification: Although we have proposed an SMT-based formal verification mechanism that can automatically detect issues such as redundancy and conflicts between policies, the completeness and efficiency of verification still face challenges for complex policy combinations. Specifically, when the policy set contains a large number of nested quantified constraints (such as multi-level role inheritance combined with ABAC conditions), the search space of the SMT solver may grow exponentially, leading to significantly extended verification time. In our experiments, approximately 3.7% of verification queries failed to complete due to timeout (exceeding 300 seconds). How to balance expressiveness and verifiability, and how to combine with other formal methods (such as model checking) to provide stronger security guarantees, all require further theoretical analysis and engineering practice.

Stability of Graph RAG policy generation: This paper has made preliminary attempts in Graph RAG-assisted policy generation and achieved good results, but the quality of the generated policies is not yet stable enough. In the ablation experiments, the correctness of the Graph RAG variant is 89.2%, meaning that approximately 10.8% of the generated policies have problems. Specifically, common types of errors include: syntax errors (such as misspelled attribute names and incorrect expression formats), semantic unreasonableness (such as granting overly broad permissions), and inconsistencies with existing policies. These problems mainly stem from the uncertainty of LLMs

when handling domain-specific syntax, and the insufficient coverage of certain policy patterns in the training data. In the future, we plan to introduce a larger-scale policy graph knowledge base, while optimizing the policy quality evaluation and feedback mechanism (such as introducing rule-based post-processing steps to correct common syntax errors), to further enhance Graph RAG's policy generation capability.

B. Research Ethics

This research strictly followed research ethics guidelines in its design and implementation to ensure that the research activities did not have a negative impact on participants or related entities. Especially in the research and development of authorization policies, we considered potential ethical issues, such as the protection of user privacy rights and the assurance of data security. All data collection and processing activities were based on the informed consent of users and complied with data protection regulations. This research has been reviewed and approved by the Institutional Review Board (IRB) of the organization where it was conducted. In addition, we paid special attention to the security of the proposed authorization framework, ensuring that it itself does not become a stepping stone for attackers. Through formal verification and other means, we minimized the new vulnerabilities and attack surfaces it might introduce.

VI. Conclusion

This paper proposes PML, an easy-to-use and efficient graph-model-based authorization policy language, and Casbin, an authorization framework, to address the increasingly complex access control requirements in cloud environments. Casbin adopts PML, a graph-based policy language, modeling authorization as a relational graph, and leverages optimization techniques from the field of graph computing to achieve high-speed policy matching. At the same time, Casbin introduces SMT solvers for formal verification of policies, and uses Graph RAG to assist in generating high-quality authorization policies. Large-scale real-world experiments show that Casbin significantly outperforms existing methods in terms of functional expressiveness, policy evaluation performance, verifiability, and policy authoring usability. In the future, we plan to further optimize Casbin's distributed execution performance, support larger-scale authorization requests, and explore using AI techniques such as reinforcement learning for automatic policy optimization.

References

[1] R. S. Sandhu, E. J. Coyne, H. L. Feinstein, and C. E. Youman, "Role-based access control models," *Computer*, vol. 29, no. 2, pp. 38–47, 1996.

[2] D. F. Ferraiolo, R. Sandhu, S. Gavrila, D. R. Kuhn, and R. Chandramouli, "Proposed NIST standard for role-based access control," *ACM Trans. Inf. System Secur.*, vol. 4, no. 3, pp. 224–274, 2001.

[3] B. Tang and R. Sandhu, "Extending openstack access control with domain trust," in *Network and System Security*. Berlin, Germany: Springer, 2014, pp. 54–69.

[4] V. C. Hu, D. R. Kuhn, D. F. Ferraiolo, and J. Voas, "Attribute-based access control," *Computer*, vol. 48, no. 2, pp. 85–88, 2015.

[5] R. Pang et al., "Zanzibar: Google's consistent, global authorization system," in *Proc. USENIX Annu. Tech. Conf.*, 2019, pp. 33–46.

[6] J. W. Cutler et al., "Cedar: A new language for expressive, fast, safe, and analyzable authorization," in *Proc. ACM Program. Languages*, vol. 8, no. OOPSLA 1, pp. 670–697, 2024.

[7] T. L. Foundation, "OpenFGA: A high performance and flexible authorization permission engine built for developers and inspired by Google zanzibar," 2024. Accessed: Jun. 26, 2026. [Online]. Available: https://openfga.dev/

[8] AuthZed, "SpiceDB: Open source, Google zanzibar-inspired database for fine-grained authorization," 2021. Accessed: Jun. 26, 2026. [Online]. Available: https://github.com/authzed/spicedb,

[9] C. N. C. Foundation, "Opa rego: Policy-based control for cloud native environments," 2022. Jun. 26, 2026. [Online]. Available: https://www.openpolicyagent.org/

[10] OASIS, "Extensible access control markup language (XACML) version 3.0," 2013. Accessed: Jun. 26, 2026. [Online]. Available: https://docs.oasis-open.org/xacml/3.0/xacml-3.0-core-spec-os-en.html

[11] C. N. C. Foundation, "Open policy agent documentation: Linear fragment," 2023. Accessed: Jun. 26, 2026. [Online]. Available: https://www.openpolicyagent.org/docs/policy-performance#linear-fragment,

[12] Y. Luo, Q. Shen, and Z. Wu, "PML: An interpreter-based access control policy language for web services," 2019, *arXiv:1903.09756*.

[13] Y. Luo, Q. Shen, and Z. Wu, "PERM: Streamlining cloud authorization with flexible and scalable policy enforcement," *IEEE Trans. Inf. Forensics Secur.*, vol. 20, no. 7, pp. 8481–8496, 2025.

[14] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, vol. 33, pp. 9459–9474.

[15] N. Damianou, N. Dulay, E. Lupu, and M. Sloman, "The ponder policy specification language," in *Policies for Distributed Systems and Networks*. Berlin, Germany: Springer, 2001, pp. 18–38.

[16] C. Ribeiro, A. Zúquete, P. Ferreira, and P. Guedes, "SPL: An access control language for security policies and complex constraints," in *Proc. Netw. Distrib. System Secur. Symp.*, 2001, pp. 1–19.

[17] Cerbos.dev, "Cerbos: Open core, language-agnostic, scalable authorization solution," 2021. Jun. 26, 2026. [Online]. Available: https://www.cerbos.dev/

[18] R. Wu, X. Zhang, G.-J. Ahn, H. Sharifi, and H. Xie, "ACaaS: Access control as a service for IaaS cloud," in *Proc. Int. Conf. Social Comput.*, 2013, pp. 423–428.

[19] X. Jin, R. Krishnan, and R. Sandhu, "Role and attribute based collaborative administration of intra-tenant cloud IAAS," in *Proc. Collaborative Comput.: Netw., Appl. Worksharing*, 2014, pp. 261–274.

[20] L. Lamport, "Paxos made simple," *ACM SIGACT News*, vol. 32, no. 4, pp. 51–58, 2001.

[21] L. De Moura and N. Bjørner, "Z3: An efficient SMT solver," in *Proc. Int. Conf. Tools Algorithms Construction Anal. Syst.*, 2008, pp. 337–340.

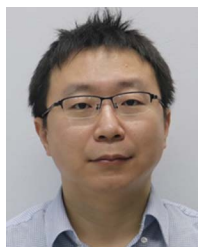
[22] E. Clarke, O. Grumberg, S. Jha, Y. Lu, and H. Veith, "Counterexample-guided abstraction refinement," in *Proc. 12th Int. Conf. Comput. Aided Verification*, Chicago, IL, USA, 2000, pp. 154–169.

[23] A. Grattafiori et al., "The Llama 3 herd of models," 2024, *arXiv:2407.21783*.

[24] S. Zhang et al., "OPT: Open pre-trained transformer language models," 2022, *arXiv:2205.01068*.

[25] E. J. Hu et al., "LoRA: Low-rank adaptation of large language models," in *Proc. Int. Conf. Learn. Representations*, 2022, pp. 1–13.

[26] IBM, "LoopBack: A highly-extensible, open-source node.js framework," 2023. Accessed: Jun. 26, 2026. [Online]. Available: https://loopback.io/



Yang Luo (Member, IEEE) received the PhD degree from Peking University, Beijing, China, in 2018. He is currently an assistant professor with the National Engineering Research Center for Software Engineering, Peking University. His research interests include cloud security, access control, software engineering, and artificial intelligence.

Qingni Shen (Senior Member, IEEE) received the PhD degree from the Institute of Software, Chinese Academy of Sciences, Beijing, China, in 2006. She is currently a professor with the School of Software and Microelectronics, Peking University, Beijing. Her research interests include operating systems and virtualization security, privacy-preserving in cloud and Big Data, and trusted computing. She is an ACM and CCF Member. She was in many international conferences, including TrustCom and SecureComm, and the chair of publicity committee or a member of program committee for ICICS in 2007, 2009, 2011, 2013, 2015, and 2017, respectively.

Zhonghai Wu received the PhD degree from Zhejiang University, Hangzhou, China, in 1997. He was a postdoctoral and senior research fellow with the Institute of Computer Science and Technology, Peking University, Beijing, China. He is currently the executive president with the School of Software and Microelectronics, Peking University. His research interests include context-aware services, embedded software and systems, cloud services and security, open innovation. He has been involved in the research and application development of distributed systems and multimedia applications.